arXiv:2407.04841v2 [cs.CL] 13 Feb 2025

Associative Recurrent Memory Transformer

Ivan Rodkin[1] **Yuri Kuratov**[2,1] **Aydar Bulatov**[1] **Mikhail Burtsev**[3]

[1]Neural Networks and Deep Learning Lab, MIPT, Dolgoprudny, Russia
[2]AIRI, Moscow, Russia [3]London Institute for Mathematical Sciences, London, UK
{rodkin.id,yurii.kuratov,bulatov.as}@phystech.edu, mb@lims.ac.uk

Abstract

This paper addresses the challenge of creating a neural architecture for very long sequences that requires constant time for processing new information at each time step. Our approach, Associative Recurrent Memory Transformer (ARMT), is based on transformer self-attention for local context and segment-level recurrence for storage of task specific information distributed over a long context. We demonstrate that ARMT outperfors existing alternatives in associative retrieval tasks and sets a new performance record in the recent BABILong multi-task long-context benchmark by answering single-fact questions over 50 million tokens with an accuracy of 79.9%. The source code for training and evaluation is available on github.

1. Introduction

Memory plays a crucial role in creating models capable of processing extremely long contexts and utilizing remote past information. Starting from RNNs and evolving through LSTM [13] and Memory Networks [24, 28], we are now in the era of Transformer-based [27] Large Language Models [2, 17, 26]. Various methods for extending transformers context length have emerged [4, 19, 31], including approaches based on transformer segment-level recurrence [3, 5, 6, 20] and novel architectures that combine the efficiency of transformer parallelization during training with recurrence at inference [7, 8, 10, 11, 18]. Alternatively, Retrieval-Augmented Generation (RAG) focuses on retrieving information from external storage [1, 12, 23] or self-retrieving from past inputs [21, 29]. However, retrieval fails on complex tasks that require reasoning over multiple pieces of information [16].

In this work we propose the Associative Recurrent Memory Transformer (ARMT) as an extension of the segment-level recurrent model RMT [3] with associative memory. Compared to RWKV [18] and Mamba [10], which use association-based techniques, ARMT benefits from full local self-attention and has constant time and space complexity of processing new segment, similar to RMT. To study ARMT performance we use the BABILong [16] benchmark, because it allows to generate test samples up to 50 million tokens and beyond, compared to other methods. Additionally, we use Associative Retrieval task with multiple key-value pairs to estimate memory capacity of models.

Main contributions of this work include: **(1)** a novel ARMT architecture for long context processing with segment-level recurrence and associative memory; **(2)** demonstration that ARMT outcompetes existing memory based models like RMT [3] and Mamba [10] on associative retrieval and long context processing tasks, achieving 80% accuracy of single fact QA on unprecedented input

© I. Rodkin, Y. Kuratov, A. Bulatov & M. Burtsev.

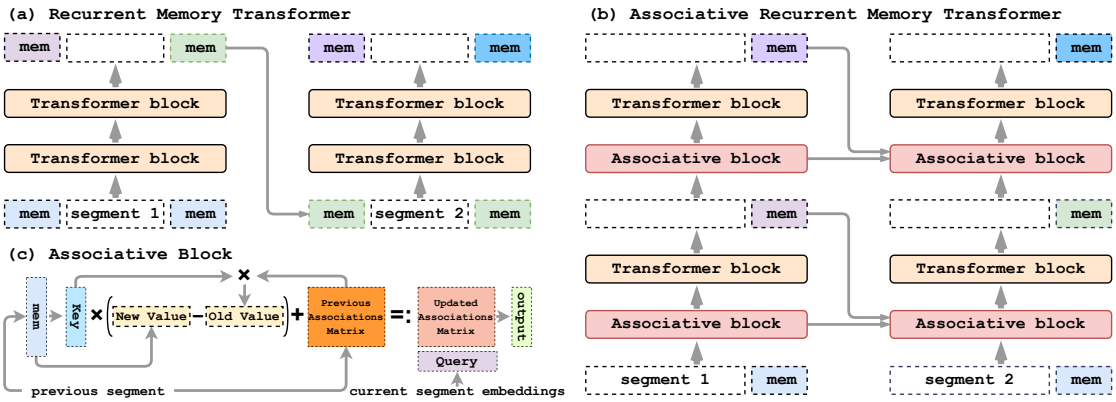


Figure 1: **ARMT augments the transformer's layers with associative memory.** (a) RMT architecture. (b) ARMT adds associative memory processing to each layer. (c) Associative memory is updated with layerwise memory representations.

size of 50 million tokens; **(3)** an original method to evaluate memory capacity in associative retrieval task.

2. Associative Recurrent Memory Transformer

We extend RMT [3](Fig. 1a) by addition of layerwise associative memory A_s^l over segmented input X_s^l (Fig. 1b). At every input segment s for each layer l memory tokens M_{s-1}^{l+1} generated for preceding segment are added to A_s^l (Fig. 1c) used to update input sequence and memory embeddings:

$$[X_s^{l+1}; M_s^{l+1}] = TrBlock(AssocBlock([X_s^l; M_s^l], A_s^l)); \quad A_s^l = MemUpdate(A_{s-1}^l; M_{s-1}^{l+1}).$$

The mechanism of associative block (Fig. 1c) is similar to linear transformers [15], but attends only to special memory tokens and is calculated differenty. After each segment, memory tokens are converted to keys and values via linear mapping and then stored in quasi-linear key-value memory [22] using non-linearity ϕ. Given a memory token $m_i \in M_s^{l+1}$, we calculate the keys, values, and importance scalars β_i. We then recall the previous association $\overline{v}_i$ with this key, add the new value v_i to the memory, erase the previous value $\overline{v}_i$ associated with k_i, and update the normalization vector.

$$k_i, v_i = W_K m_i, W_V m_i; \quad \beta_i = \sigma(W_\beta m_i); \quad A_0^l = \mathbf{0}; \quad z_0^l = \mathbf{0}; \tag{1}$$

$$\overline{v}_i = \frac{A_{s-1}^l \phi(k_i)}{(z_{s-1})^T \phi(k_i)}; \quad \gamma_i = 1 - \frac{(z_{s-1})^T \phi(k_i)}{\|\phi(k_i)\|^2}; \tag{2}$$

$$A_s^l = A_{s-1}^l + \sum_i \beta_i (v_i - \overline{v}_i) \otimes \phi(k_i); \quad z_s^l = z_{s-1}^l + \sum_i \gamma_i \phi(k_i). \tag{3}$$

Once we updated A_s^l with information from previous segment, we recall an association y_j for a token x_j. Associations y_j for each token in the segment are then passed to the next transformer layer:

$$q_j = W_Q x_j; \quad y_j = \frac{A_s^l \phi(q_j)}{(z_s^l)^T \phi(q_j)}. \tag{4}$$

For the non-linearity function ϕ, we used the proposed in [22] DPFP-3 function because, in this paper, it has shown significant superiority over other methods, which is also consistent with our findings.

Note that without γ_i ($\gamma_i = 1$) this approach suffers from catastrophic forgetting on some tasks. The reason is that while we erase the information $\overline{v}_i$ from the A-matrix, the corresponding keys

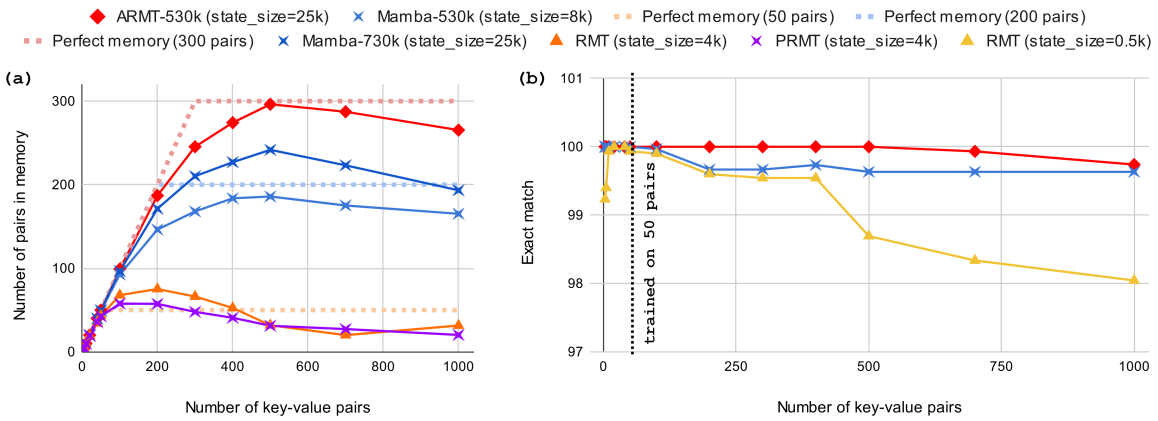


Figure 2: **ARMT demonstrates strong performance on associative memory tasks. (a)** The estimated number of pairs, stored in memory after processing the context with key-value pairs. **(b)** ARMT is more accurate at operations in memory. Being trained only on 50 key-value pairs from Associative Retrieval Rewrite task, ARMT performs accurate even on 500 memory updates. So the observed generalization factor is 10 (500 pairs / 50 pairs). All data are averaged over 3 runs except RMT and PRMT with 2 runs.

remain in the normalization vector z_s. As shown in our experiments (Fig. 4(a)), this problem becomes significant when performing hundreds of erase-insert operations with associative memory. To overcome this, we propose to take into account the previous keys in z_s when updating it (details are in Appendix F.1).

To determine which part contributes the most to ARMT performance, we also studied RMT with layerwise recurrent memory without associative memory block (Parallel Memory RMT, or PRMT; see Fig. 5 in Appendix F.2).

3. Evaluation of Associative Retrieval and Long Context Memory Retention

We test memory capacity of ARMT in comparison to recent computationally efficient long-context models Mamba [10] and RMT [3] on the following two variants of associative retrieval.[1]

Remember task requires memorization of *all* key-value pairs with unique keys from the prior context with subsequent recalling a value corresponding to one of the keys (Fig. 2a). We estimate the total number of key-value pairs stored in memory, based on the exact-match metric (details are in Appendix B). Since ARMT has the same recurrent memory size as Mamba, calculated as the number of floats in recurrent states, it can be concluded that ARMT makes better use of its internal associative memory. Both ARMT and Mamba outperform RMT on this task. PRMT does not improve RMT performance (Fig. 2a). This indicates that the associative memory plays a critical role in ARMT performance compared to RMT. Additionally, we ablated ARMT on normalization correction, as detailed in Appendix F.

In **Rewrite task** the keys are not unique and the goal is to recall the *latest* value that corresponds to one of the keys from the prior context. This task evaluates the model's ability to dynamically change the memory storage. The results, shown in Fig. 2b, indicate that ARMT is robust to the number of memory rewrite operations, while RMT and Mamba experience slight degradation after exceeding their training lengths. Notably, ARMT maintains perfect memory recall on lengths exceeding 10 times those used in training.

1. We did our best but failed to train RWKV model for associative retrieval and BABILong benchmarks, Appendix J.

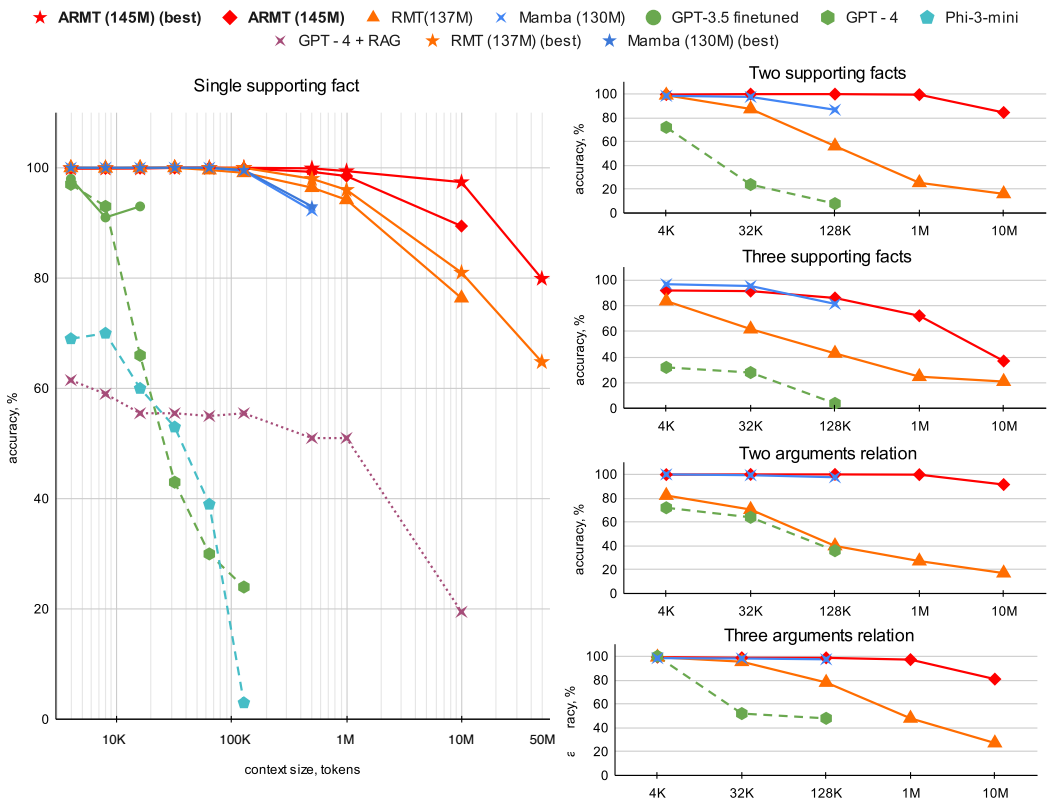


Figure 3: **ARMT sets a record in long-context processing with reasonable performance on 50 million tokens.** Accuracy of models on different lengths from Babilong benchmark: panels a-e represent QA1-5 tasks.

We augment the GPT-2 (137M) model with ARMT to solve the BABILong tasks from a recently introduced benchmark for long context processing in a question-answering form [16]. To answer BABILong questions correctly, models have to find multiple relevant facts distributed across long natural contexts with distractor facts, and combine information from relevant facts. We use the exact match metric to evaluate models' performance. As shown in Fig. 3, ARMT outperforms the competitors in the majority of tasks, especially on long sequences. Being trained on 16k tokens only, it strongly performs up to 50 million tokens on QA1 single supporting fact (Fig. 3a) and up to 10 million tokens on more complex tasks requiring multi-hop reasoning (Fig. 3b-e). We observe 60x length-generalization on these tasks (1M / 16k), while Mamba has 8x length-generalization (128k / 16k). For Mamba-130m we consider only the lengths up to 128k due to its implementation limitations (see Appendix H, and Appendices C and D for training details).

4. Conclusion

In this work, we propose and evaluate recurrent memory transformer augmented with associative memory mechanism for long-context processing and find that it scales up to an unprecedented 50 million tokens on the BABILong benchmark. ARMT architecture adds an associative memory mechanism for segment-level recurrent model RMT. Based on our evaluation on associative retrieval tasks ARMT demonstrates significant advantage in memory capacity and generalisation compared to

Mamba and RMT. On BABILong benchmark ARMT dominates alternatives on medium sizes up to 500K tokens and the only approach with high performance across all five tasks in the range of 500K-10M tokens.

We conclude that ARMT holds great promise for long-range tasks because of its improved memory capacity, its ability to efficiently handle large numbers of rewrite operations with memory, its ability to extract only relevant information from memory during inference, and its generalization to much longer sequences than it was trained on. We also assume that the ARMT can be used for language modeling (Appendices G and K). Despite the current results, we believe there is potential to enhance its performance on LM task through further research and optimization.

Since all of the results in this study are obtained on relatively small (137M) models, we also assume that the scaling of our methodology and its combination with other techniques can reveal the significant potential for modern large language models. We believe that investigating the properties of recurrent associative memory remains an exciting area of research.

Acknowledgements

We are thankful to SberDevices for granting us access to additional computational resources. This work was supported by a grant for research centers, provided by the Analytical Center for the Government of the Russian Federation in accordance with the subsidy agreement (agreement identifier 000000D730324P540002) and the agreement with the Moscow Institute of Physics and Technology dated November 1, 2021 No. 70-2021-00138.

References

[1] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, et al. Improving language models by retrieving from trillions of tokens. In *International conference on machine learning*, pages 2206–2240. PMLR, 2022.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Aydar Bulatov, Yuri Kuratov, and Mikhail S. Burtsev. Recurrent memory transformer, 2022.

[4] Yukang Chen, Shengju Qian, Haotian Tang, Xin Lai, Zhijian Liu, Song Han, and Jiaya Jia. Longlora: Efficient fine-tuning of long-context large language models. In *The Twelfth International Conference on Learning Representations*, 2023.

[5] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. Adapting language models to compress contexts, 2023.

[6] Zihang Dai, Zhilin Yang, Yiming Yang, Jaime G Carbonell, Quoc Le, and Ruslan Salakhutdinov. Transformer-xl: Attentive language models beyond a fixed-length context. In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 2978–2988, 2019.

[7] Soham De, Samuel L. Smith, Anushan Fernando, Aleksandar Botev, George Cristian-Muraru, Albert Gu, Ruba Haroun, Leonard Berrada, Yutian Chen, Srivatsan Srinivasan, Guillaume Desjardins, Arnaud Doucet, David Budden, Yee Whye Teh, Razvan Pascanu, Nando De Freitas, and Caglar Gulcehre. Griffin: Mixing gated linear recurrences with local attention for efficient language models, 2024.

[8] Daniel Y. Fu, Tri Dao, Khaled K. Saab, Armin W. Thomas, Atri Rudra, and Christopher Ré. Hungry hungry hippos: Towards language modeling with state space models, 2023.

[9] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. The pile: An 800gb dataset of diverse text for language modeling, 2020.

[10] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces, 2023.

[11] Albert Gu, Karan Goel, and Christopher Re. Efficiently modeling long sequences with structured state spaces. In *International Conference on Learning Representations*, 2021.

[12] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Mingwei Chang. Retrieval augmented language model pre-training. In *International conference on machine learning*, pages 3929–3938. PMLR, 2020.

[13] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Comput.*, 9 (8):1735–1780, November 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL https://doi.org/10.1162/neco.1997.9.8.1735.

[14] Samy Jelassi, David Brandfonbrener, Sham M. Kakade, and Eran Malach. Repeat after me: Transformers are better than state space models at copying, 2024.

[15] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention, 2020.

[16] Yuri Kuratov, Aydar Bulatov, Petr Anokhin, Dmitry Sorokin, Artyom Sorokin, and Mikhail Burtsev. In search of needles in a 11m haystack: Recurrent memory finds what llms miss, 2024.

[17] OpenAI, Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, Suchir Balaji, Valerie Balcom, Paul Baltescu, Haiming Bao, Mohammad Bavarian, Jeff Belgum, Irwan Bello, Jake Berdine, Gabriel Bernadett-Shapiro, Christopher Berner, Lenny Bogdonoff, Oleg Boiko, Madelaine Boyd, Anna-Luisa Brakman, Greg Brockman, Tim Brooks, Miles Brundage, Kevin Button, Trevor Cai, Rosie Campbell, Andrew Cann, Brittany Carey, Chelsea Carlson, Rory Carmichael, Brooke Chan, Che Chang, Fotis Chantzis, Derek Chen, Sully Chen, Ruby Chen, Jason Chen, Mark Chen, Ben Chess, Chester Cho, Casey Chu, Hyung Won Chung, Dave Cummings, Jeremiah Currier, Yunxing Dai, Cory Decareaux, Thomas Degry, Noah Deutsch, Damien Deville, Arka Dhar, David Dohan, Steve Dowling, Sheila Dunning, Adrien Ecoffet, Atty Eleti, Tyna Eloundou, David Farhi, Liam Fedus, Niko Felix, Simón Posada Fishman, Juston Forte, Isabella Fulford, Leo Gao, Elie Georges, Christian Gibson, Vik Goel, Tarun Gogineni, Gabriel Goh, Rapha Gontijo-Lopes,

Jonathan Gordon, Morgan Grafstein, Scott Gray, Ryan Greene, Joshua Gross, Shixiang Shane Gu, Yufei Guo, Chris Hallacy, Jesse Han, Jeff Harris, Yuchen He, Mike Heaton, Johannes Heidecke, Chris Hesse, Alan Hickey, Wade Hickey, Peter Hoeschele, Brandon Houghton, Kenny Hsu, Shengli Hu, Xin Hu, Joost Huizinga, Shantanu Jain, Shawn Jain, Joanne Jang, Angela Jiang, Roger Jiang, Haozhun Jin, Denny Jin, Shino Jomoto, Billie Jonn, Heewoo Jun, Tomer Kaftan, Łukasz Kaiser, Ali Kamali, Ingmar Kanitscheider, Nitish Shirish Keskar, Tabarak Khan, Logan Kilpatrick, Jong Wook Kim, Christina Kim, Yongjik Kim, Jan Hendrik Kirchner, Jamie Kiros, Matt Knight, Daniel Kokotajlo, Łukasz Kondraciuk, Andrew Kondrich, Aris Konstantinidis, Kyle Kosic, Gretchen Krueger, Vishal Kuo, Michael Lampe, Ikai Lan, Teddy Lee, Jan Leike, Jade Leung, Daniel Levy, Chak Ming Li, Rachel Lim, Molly Lin, Stephanie Lin, Mateusz Litwin, Theresa Lopez, Ryan Lowe, Patricia Lue, Anna Makanju, Kim Malfacini, Sam Manning, Todor Markov, Yaniv Markovski, Bianca Martin, Katie Mayer, Andrew Mayne, Bob McGrew, Scott Mayer McKinney, Christine McLeavey, Paul McMillan, Jake McNeil, David Medina, Aalok Mehta, Jacob Menick, Luke Metz, Andrey Mishchenko, Pamela Mishkin, Vinnie Monaco, Evan Morikawa, Daniel Mossing, Tong Mu, Mira Murati, Oleg Murk, David Mély, Ashvin Nair, Reiichiro Nakano, Rajeev Nayak, Arvind Neelakantan, Richard Ngo, Hyeonwoo Noh, Long Ouyang, Cullen O'Keefe, Jakub Pachocki, Alex Paino, Joe Palermo, Ashley Pantuliano, Giambattista Parascandolo, Joel Parish, Emy Parparita, Alex Passos, Mikhail Pavlov, Andrew Peng, Adam Perelman, Filipe de Avila Belbute Peres, Michael Petrov, Henrique Ponde de Oliveira Pinto, Michael, Pokorny, Michelle Pokrass, Vitchyr H. Pong, Tolly Powell, Alethea Power, Boris Power, Elizabeth Proehl, Raul Puri, Alec Radford, Jack Rae, Aditya Ramesh, Cameron Raymond, Francis Real, Kendra Rimbach, Carl Ross, Bob Rotsted, Henri Roussez, Nick Ryder, Mario Saltarelli, Ted Sanders, Shibani Santurkar, Girish Sastry, Heather Schmidt, David Schnurr, John Schulman, Daniel Selsam, Kyla Sheppard, Toki Sherbakov, Jessica Shieh, Sarah Shoker, Pranav Shyam, Szymon Sidor, Eric Sigler, Maddie Simens, Jordan Sitkin, Katarina Slama, Ian Sohl, Benjamin Sokolowsky, Yang Song, Natalie Staudacher, Felipe Petroski Such, Natalie Summers, Ilya Sutskever, Jie Tang, Nikolas Tezak, Madeleine B. Thompson, Phil Tillet, Amin Tootoonchian, Elizabeth Tseng, Preston Tuggle, Nick Turley, Jerry Tworek, Juan Felipe Cerón Uribe, Andrea Vallone, Arun Vijayvergiya, Chelsea Voss, Carroll Wainwright, Justin Jay Wang, Alvin Wang, Ben Wang, Jonathan Ward, Jason Wei, CJ Weinmann, Akila Welihinda, Peter Welinder, Jiayi Weng, Lilian Weng, Matt Wiethoff, Dave Willner, Clemens Winter, Samuel Wolrich, Hannah Wong, Lauren Workman, Sherwin Wu, Jeff Wu, Michael Wu, Kai Xiao, Tao Xu, Sarah Yoo, Kevin Yu, Qiming Yuan, Wojciech Zaremba, Rowan Zellers, Chong Zhang, Marvin Zhang, Shengjia Zhao, Tianhao Zheng, Juntang Zhuang, William Zhuk, and Barret Zoph. Gpt-4 technical report, 2024.

[18] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, Stella Biderman, Huanqi Cao, Xin Cheng, Michael Chung, Matteo Grella, Kranthi Kiran GV, Xuzheng He, Haowen Hou, Jiaju Lin, Przemyslaw Kazienko, Jan Kocon, Jiaming Kong, Bartlomiej Koptyra, Hayden Lau, Krishna Sri Ipsit Mantri, Ferdinand Mom, Atsushi Saito, Guangyu Song, Xiangru Tang, Bolun Wang, Johan S. Wind, Stanislaw Wozniak, Ruichong Zhang, Zhenyuan Zhang, Qihang Zhao, Peng Zhou, Qinghua Zhou, Jian Zhu, and Rui-Jie Zhu. Rwkv: Reinventing rnns for the transformer era, 2023.

[19] Bowen Peng, Jeffrey Quesnelle, Honglu Fan, and Enrico Shippole. Yarn: Efficient context window extension of large language models. In *The Twelfth International Conference on*

Learning Representations, 2023.

[20] Jack W Rae, Anna Potapenko, Siddhant M Jayakumar, Chloe Hillier, and Timothy P Lillicrap. Compressive transformers for long-range sequence modelling. *arXiv preprint*, 2019. URL https://arxiv.org/abs/1911.05507.

[21] Ohad Rubin and Jonathan Berant. Long-range language modeling with self-retrieval. *arXiv preprint arXiv:2306.13421*, 2023.

[22] Imanol Schlag, Kazuki Irie, and Jürgen Schmidhuber. Linear transformers are secretly fast weight programmers, 2021.

[23] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Rich James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. Replug: Retrieval-augmented black-box language models. *arXiv preprint arXiv:2301.12652*, 2023.

[24] Sainbayar Sukhbaatar, Arthur Szlam, Jason Weston, and Rob Fergus. End-to-end memory networks, 2015.

[25] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models, 2023.

[26] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

[27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is All you Need. In *Advances in neural information processing systems*, pages 5998–6008, 2017. URL http://papers.nips.cc/paper/7181-attention-is-all-you-need.

[28] Jason Weston, Sumit Chopra, and Antoine Bordes. Memory networks. In Yoshua Bengio and Yann LeCun, editors, *3rd International Conference on Learning Representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, Conference Track Proceedings*, 2015. URL http://arxiv.org/abs/1410.3916.

[29] Yuhuai Wu, Markus Norman Rabe, DeLesley Hutchins, and Christian Szegedy. Memorizing transformers. In *International Conference on Learning Representations*, 2022. URL https://openreview.net/forum?id=TrjbxzRcnf-.

[30] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks, 2023.

[31] Peitian Zhang, Zheng Liu, Shitao Xiao, Ninglu Shao, Qiwei Ye, and Zhicheng Dou. Soaring from 4k to 400k: Extending llm's context with activation beacon, 2024.

Appendix A. Related Work

AutoCompressor [5] is a strategy that stacks memory to minimize information loss at the price of quadratic computation cost.

Recurrent Memory Transformers Recent challenges in long-context processing tasks demonstrated recurrent memory superiority over attention mechanism [16, 31] (Fig. 1 (a)). It was shown that this type of memory performs well even in contexts of size 11M [16]. But still, this memory has some issues with capacity and training. Capacity remains limited as the suggested memory states are limited to a small number of memory tokens. The training is still challenging, as the whole training process requires backpropagation through time for hundreds of layers. Our approach is supposed to mitigate these problems by leveraging the association matrix as a connector for different segments. In contrast to RMT, it has different parameters for memory (linear attention projections described in Section 2) and makes this memory hierarchical by creating different association matrices for different layers.

Context explosion prevention In the attention sinks paper [30], authors demonstrated the need for some sinking tokens for attention for efficient extrapolation in long contexts. Recurrent Memory in RMT [3] as well as our model successfully perform this function. In RMT the memory tokens can act as attention sinks while in our model the very association matrix can play this role.

The ARMT can also be thought of as a kind of compressed-memory RMT [3], that attends to all previous memory tokens with layerwise memory.

Recurrent Sequence Models The problem of the quadratic cost of attention mechanism led to the development of recurrent architectures with transformer-like performance [7]. The vast majority of them are at least related to the State-Space Models (SSMs) [8, 10, 18, 25]. Despite comparable performance with transformers on LM tasks, SSMs are known to be less efficient in memorization tasks, especially when the question is asked after the information [14]. Our model performs well even on these types of tasks, because it has the large and flexible storage for keeping the associations in memory, simultaneously having the direct access to the local context via the vanilla attention.

Appendix B. Memory capacity estimation

Theorem:

Given:

$$\text{exact_match} = \alpha; \quad n = \text{number of pairs}; \quad v = \text{number of possible values}$$

Then the number of memorized pairs can be estimated with the formula:

$$k = \frac{nv\alpha - n}{v - 1}$$

Proof:

We can precisely predict the associated value if we remember k pairs and then extract the key from these pairs. We output the random value if we obtain the key from any other pair. As a result, the following is the mathematical expectation of an exact match:

$$\alpha = \frac{k}{n} \cdot 1 + \frac{n-k}{n} \cdot \frac{1}{v} = \frac{k}{n}(1 - \frac{1}{v}) + \frac{1}{v} = \frac{k(v-1)+n}{nv}$$

$$k = \frac{nv\alpha - n}{v - 1}$$

Additionally:

$$k = n\alpha\frac{v - \frac{1}{\alpha}}{v - 1} = n\alpha\frac{v\alpha - 1}{v\alpha - \alpha} = n\frac{v\alpha - 1}{v - 1}$$

Appendix C. Curriculum learning

We train all models with curriculum learning. This means we incrementally increase the complexity of the task during the training. In particular, we train all models on short sequences first and then increase the length of the sequences until it reaches the maximum length (16k tokens for babilong experiments, 200 pairs for Associative Retrieval Remember, 50 pairs for Associative Retireval Rewrite, and 1024 tokens for language modeling experiments (8 segments, 128 each)).

Appendix D. Babilong training details

We consider segments of size 512 for RMT and ARMT to process the long sequences. The curriculum learning process uses the following number of sequences consecutively: 2, 3, 5, 8, 16, 32. So the training ends when we finish training on 32 segments, 512 tokens each. We also randomly sample the number of segments during training, as we find it helps the model generalize better.

Appendix E. Associative Retrieval training details

Due to the task's simplicity and training efficiency, we are considering small models (about 500k parameters each) for the Associative Retrieval dataset studies. Every model that we compare has four layers. 128 is the hidden dimension. If the memory dimension parameter (state size in Mamba and memory dimension in ARMT) is present in the model, it is assumed to be 32; if the contrary is not indicated.

Moreover, if the model supports segmentation (like RMT and ARMT), we use different segments for different key-value pairs. Thus, if we have, for instance, 200 pairs, 200 segments are passed through the model, and after that, in the 201st segment, we expect the model to generate the value. Both keys and values consist of several integers from 0 to 15.

We also use the curriculum with the following number of key-value pairs: 1, 2, 3, 5, 10, 20, 40, 50, and 200. We increase the key size if necessary, so the final key size for remember task is 3 (so we have 16^3 unique keys) and for rewrite task it remains 1 (16 unique keys). For the **Remember task**, we also consider sampling different numbers of pairs during training for better generalization.

Appendix F. Ablation

F.1. Gamma-correction

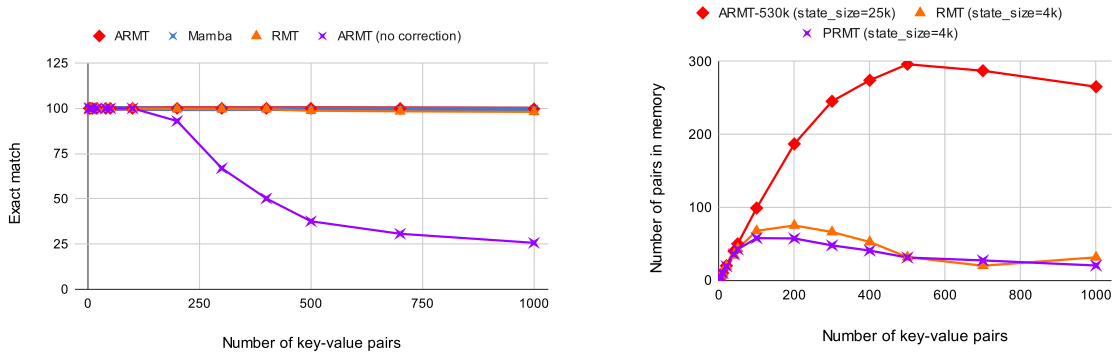
Due to an improper normalization vector z update, the proposed fast-weights technique [22] (also known as delta-rule) does not have the length generalization (Fig. 4(a)). The information in the association matrix A is erased, but not from z, which is the source of the issue.

	64k	**128k**	**500k**	**1M**	**10M**	**50M**
QA1 – SINGLE SUPPORTING FACT						
GPT-4 (Few-shot)	30.0	24.0	-	-	-	-
GPT-4 + RAG (Few-shot)	50.0	56.0	50.0	56.0	16.0	-
RMT (137M)	99.6	99.1	96.4	94.2	76.4	- (64,8*)
RMT-R	99.7	99.5	97.5	97.4	86.0	-
Mamba (130M)	**100**$_{\pm 0.0}$	99.5$_{\pm 0.2}$	92.3$_{\pm 1.1}$	-	-	-
ARMT (145M)	**100**$_{\pm 0.0}$	**99.9**$_{\pm 0.2}$	**99.3**$_{\pm 0.9}$	**98.5**$_{\pm 1.0}$	89.4$_{\pm 8.1}$	49,6$_{\pm 40.4}$ (79,9*)
QA2 – TWO SUPPORTING FACTS						
GPT-4 (Few-shot)	4.0	8.0	-	-	-	-
RMT (137M)	72.7	56.3	32.0	25.5	16.2	-
RMT-R	71,6	54,9	31.8	26.3	13.0	-
Mamba (130M)	95.0$_{\pm 4.2}$	86.7$_{\pm 6.2}$	-	-	-	-
ARMT (145M)	**100**$_{\pm 0.0}$	**99.8**$_{\pm 0.2}$	**99.4**$_{\pm 0.3}$	**99.4**$_{\pm 0.3}$	**84.4**$_{\pm 4.0}$	-
QA3 – THREE SUPPORTING FACTS						
GPT-4 (Few-shot)	12.0	4.0	-	-	-	-
RMT (137M)	51.9	42.9	25.9	24.8	21.0	-
RMT-R	52.9	41.9	25.5	22.2	16.4	-
Mamba (130M)	**91.8**$_{\pm 0.3}$	**81.4**$_{\pm 1.0}$	-	-	-	-
ARMT (145M)	**90.4**$_{\pm 2.2}$	**86.0**$_{\pm 4.8}$	**79.7**$_{\pm 10.8}$	**72.1**$_{\pm 14.2}$	**37.0**$_{\pm 10.3}$	-
QA4 – TWO ARG RELATIONS						
GPT-4 (Few-shot)	20.0	36.0	-	-	-	-
RMT (137M)	51.2	40.0	29.4	27.3	17.2	-
RMT-R	58.8	50.1	32.1	26.0	14.0	-
Mamba (130M)	99.7$_{\pm 0.2}$	**97.6**$_{\pm 2.8}$	-	-	-	-
ARMT (145M)	**100**$_{\pm 0.1}$	**100**$_{\pm 0.1}$	**99.9**$_{\pm 0.2}$	**99.8**$_{\pm 0.3}$	**91.5**$_{\pm 1.7}$	-
QA5 – THREE ARG RELATIONS						
GPT-4 (Few-shot)	64.0	48.0	-	-	-	-
RMT (137M)	88.5	78.1	56.4	48.0	27.3	-
RMT-R	86.2	77.4	55.9	49.9	35.0	-
Mamba (130M)	**98.7**$_{\pm 0.1}$	**97.5**$_{\pm 1.1}$	-	-	-	-
ARMT (145M)	**99.0**$_{\pm 0.3}$	**98.7**$_{\pm 0.4}$	**98.4**$_{\pm 0.3}$	**97.3**$_{\pm 0.6}$	**80.9**$_{\pm 7.6}$	-

Table 1: Exact match metric on QA1-5 Babilong subsets. Each column corresponds to some constant context length. Context includes both noise sentences and facts. * The 50M exact-match on QA1 is measured on 1 best model. ARMT rows are 3 runs averaged. Mamba rows are 2 runs averaged. The metric is marked bold if its ±std interval intersects the ±std interval of the best model.

	Segmentation	Memory Capacity	Working Memory	LM	Length extrapolation
Mamba	✗	✔	✔	✔✔	✔
RMT	✔	?	✔	?	✔
ARMT	✔	✔✔	✔	?	✔✔

Table 2: Models abilities.



(*a*) (*b*)

Figure 4: (a) **γ-correction cures the quasi-linear attention memory.** Without correction, the quasi-linear attention with delta-rule struggles to extrapolate on unseen amounts of memory updates. (b) **Parallel memory doesn't solve the capacity issue.** This means that the associative memory plays an important role in increasing the capacity of the memory.

Note that z is the sum of $\phi(k_i)$. Moreover, we recall the information from previous segments using the inner product of $\phi(k_i)$ and $\phi(q_i)$. This means that for accurate recall, all $\phi(k_i)$ should be orthogonal to each other. Therefore, we can expect z to be a sum of approximately orthogonal vectors.

In this sense, we simplify our task to the task of removing the $\phi(k_i)$ from the sum of vectors orthogonal to $\phi(k_i)$, with the exception of the very $\phi(k_i)$. This means that the presence of $\phi(k_i)$ can be measured by computing the inner product between this sum and $\phi(k_i)$ and dividing it by the length of $\phi(k_i)$ (just taking an orthogonal basis component).

After the insertion of the new information into our memory, we expect this sum to include only one $\phi(k_i)$. Therefore, our γ-coefficient can be computed with the following formula:

$$\gamma_i = 1 - \frac{(z_{s-1})^T \phi(k_i)}{\|\phi(k_i)\|^2};$$

$$z_s = z_{s-1} + \sum_i \gamma_i \phi(k_i).$$

The inner product $(z_{s-1})^T \phi(k_i)$ is divided by the square of $\|\phi(k_i)\|$ because the gamma will be further multiplied by $\phi(k_i)$.

We also consider detaching the gamma during training, because it seems to converge better in this case.

F.2. Associative memory ablation

To understand, if the associative memory important for memorization tasks, we consider another architecture: Parallel-memory RMT (PRMT) Fig. 5

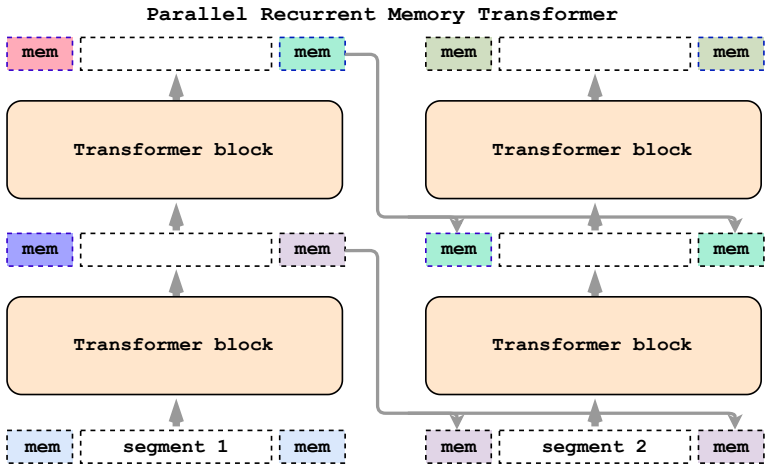


Figure 5: **Parallel recurrent memory transformer.** In contrast to RMT, in PRMT memory tokens are passed to the next segment in each layer.

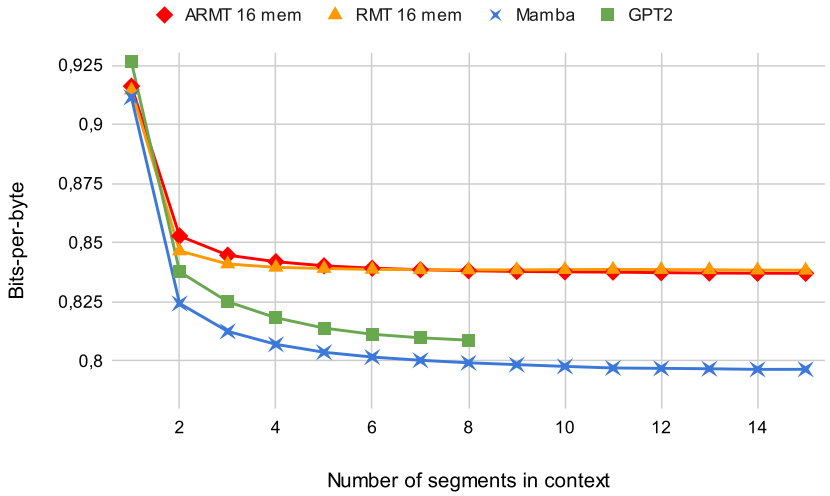


Figure 6: **ARMT performs similarly to RMT on the language modeling task.** Wikitext-103 results. The loss on each of the 128-sized segments on the test dataset (Normalized with Bits-per-byte [9]). The model is trained for a language modeling task on 8 segments, 128 tokens each. Despite the larger estimated capacity, ARMT struggles to solve the LM task well.

It is different from RMT in the hierarchical memory approach, considering the memory shifts layerwise, just like in ARMT. So this architecture can be thought of as RMT with layerwise memory, while ARMT is RMT with layerwise memory organized in association matrices.

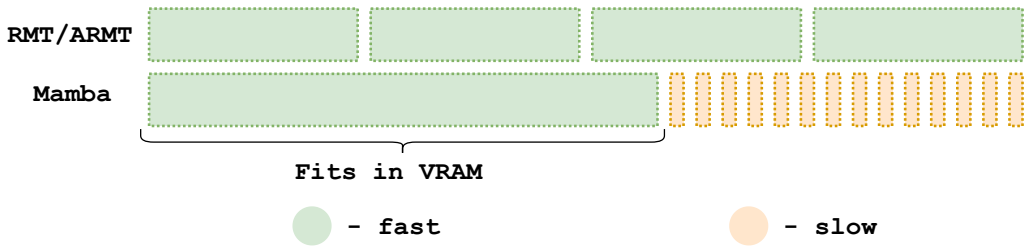


Figure 7: **Current mamba implementation allows only the first segment to be long.** Other tokens have to be processed consecutively one by one.

Appendix G. Language Modeling experiments

We utilized the Wikitext-103 dataset to train ARMT and RMT models in order to assess our architecture's performance on real texts. Next, we examined the cross-entropy losses derived from various model segments on the test dataset, as illustrated in Figure 6. In this manner, we can estimate the amount of language data that can be stored in memory.

Nevertheless, we demonstrate that despite having a larger theoretical capacity than RMT, ARMT still performs similarly to RMT in language modeling.

We use the GPT-2 model as the base model for our architecture changes. We consider the RMT and ARMT models' segment sizes to be equal to 128 tokens and train these models to solve the language modeling task on 8 segments, so in total, we train the model to autoregressively predict 1024 tokens. Then we evaluate the models' performance on each of the 15 segments of test texts (1920 tokens).

Appendix H. Why is mamba slow for long contexts?

We faced some difficulties in evaluating mamba on long-context (500k+) due to it's specific segmentation abilities shown in Figure 7.

Appendix I. Associative Retrieval sample structure

A sample of **Remember** dataset contains a concatenated context, query, and answer. The context is a set of key-value pairs (k, v), separated by a special token. All keys are sequences of tokens. Tokens in this sequence can intersect, but the whole sequence corresponding to any key is unique in this particular sample. The query is one of the keys in the context. And the answer is a value corresponding to the key from the query. Thus, we can control the number of pairs in the sample and check how many pairs fit in our memory.

This is how the dataset's sample appears:

```
<key1>:<value1>,<key2>:<value2>, <key3>:<value3>,<key2>-<value2>
```

The model is thought to be trained to produce the value following the "−" character.

Appendix J. RWKV-5 model

We also tried to train the RWKV-v5 [18] model to slove both associative retrieval and babilong tasks (training from scratch for AR tasks and finetuning 430M model for babilong task). Unfortunately, the model was training poorly and hadn't achieved reasonable scores. We used the very same parameters as for training other models. Perhaps the RWKV training process requires accurate adjustments. However, we haven't succeeded.

Appendix K. Limitations

The proposed architecture, however, has several drawbacks. The first is the lack of efficient parallel implementation: you have to process all segments consecutively. This doesn't mean that it is slow. It's fast enough to process millions of tokens in a reasonable time. However, on short and medium-length sequences (less than 300k tokens), it is much slower than, for instance, Mamba and RWKV, which have the parallel form. Moreover, as we have shown in Appendix G, it's still challenging to train ARMT to solve the language modeling task well. However, we believe that this problem is not in the very architecture, but in training process: we observe that ARMT tends to keep in memory only the last segment, and therefore struggles to extrapolate on longer sequences.